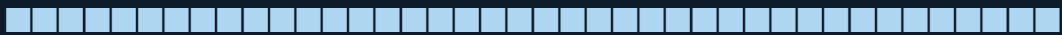


Python to AI Engineer

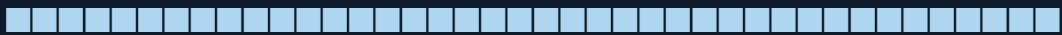
Structured LeetCode & DSA Roadmap for Freshers



■ Target: 3rd-Year CS Student → AI/ML Engineer

■ Language: Python | Platform: LeetCode

■ 120–150 Curated Problems | 4 Phases | 6-Month Plan



■ TABLE OF CONTENTS

No.	Section	Page
1.	Who Is This Roadmap For?	3
2.	Phase 1 – Python Foundations	4
3.	Phase 2 – Beginner DSA Patterns	8
4.	Phase 3 – Intermediate DSA	12
5.	Phase 4 – Advanced DSA	16
6.	Weekly Study Schedule	22
7.	Progress Tracker	23
8.	Machine Learning Parallel Roadmap	24
9.	Mathematics Guide	26
10.	Milestones & Checkpoints	27
11.	6-Month AI Engineer Preparation Plan	28

1. WHO IS THIS ROADMAP FOR?

Attribute	Details
Year	3rd-year Computer Science student
Goal	AI/ML Engineer / Software Engineer (Fresher)
Coding Language	Python
Prior LeetCode	~25 problems solved (random, no structure)
Python Level	Basic – loops, functions, lists, strings
ML Knowledge	Basic – supervised learning concepts
Daily Time	2–3 hours (1 hr DSA + 1 hr ML + 30 min review)
Target Duration	6 months

How to Use This Roadmap

- Follow phases IN ORDER – don't jump ahead.
- Aim to understand the PATTERN, not just memorise the answer.
- After solving, always revisit your solution and optimise.
- Tick off problems in the Progress Tracker on page 23.
- Run your ML study in parallel (1 hr/day) as shown in the schedule.

PHASE 1 — PYTHON FOUNDATIONS FOR PROBLEM SOLVING

Master Python's built-in data structures and standard library modules that are used daily in coding interviews. These are the building blocks for every DSA pattern.

Lists & List Operations

Why Important	Most interview problems revolve around arrays/lists. Slicing, indexing, append, pop, insert, sort – you must know these blindfolded.
Python Concepts	list(), slicing [start:stop:step], .append(), .pop(), .sort(), .reverse(), list comprehension, nested lists, enumerate(), zip()
Target Problems	8 problems

#	Problem Name	Difficulty
1	Two Sum	Easy
26	Remove Duplicates from Sorted Array	Easy
27	Remove Element	Easy
88	Merge Sorted Array	Easy
189	Rotate Array	Medium
283	Move Zeroes	Easy
485	Max Consecutive Ones	Easy
561	Array Partition	Easy

Strings

Why Important	String manipulation is tested in nearly every company. Python's rich string methods make many hard problems trivial once mastered.
Python Concepts	str methods: .split(), .join(), .strip(), .replace(), .find(), .count(), .lower(), .upper(), ord(), chr(), f-strings, string slicing
Target Problems	8 problems

#	Problem Name	Difficulty
125	Valid Palindrome	Easy
242	Valid Anagram	Easy
344	Reverse String	Easy
387	First Unique Character in String	Easy
28	Find the Index of First Occurrence	Easy
415	Add Strings	Easy
541	Reverse String II	Easy
557	Reverse Words in a String III	Easy

Dictionaries & Hash Maps

Why Important	Dictionaries give $O(1)$ lookup and are the backbone of hashing-based optimisations. Crucial for Two Sum, frequency counts, and graph adjacency lists.
Python Concepts	dict(), .get(), .items(), .keys(), .values(), defaultdict, Counter, dictionary comprehension, in operator
Target Problems	6 problems

#	Problem Name	Difficulty
1	Two Sum	Easy
217	Contains Duplicate	Easy
349	Intersection of Two Arrays	Easy
350	Intersection of Two Arrays II	Easy
383	Ransom Note	Easy
205	Isomorphic Strings	Easy

Sets

Why Important	Sets enable $O(1)$ membership testing and are perfect for duplicate removal, intersection, union, and cycle detection in graphs.
Python Concepts	set(), frozenset(), .add(), .discard(), .union(), .intersection(), .difference(), set comprehension
Target Problems	4 problems

#	Problem Name	Difficulty
217	Contains Duplicate	Easy
705	Design HashSet	Easy
128	Longest Consecutive Sequence	Medium
349	Intersection of Two Arrays	Easy

Tuples & Sorting

Why Important	Tuples are used as immutable keys in dictionaries. Custom sorting with key= is a must-have for interval, greedy, and ranking problems.
Python Concepts	tuple(), sorted(), .sort(), key=lambda, operator.itemgetter, multiple-key sort, heapq with tuples
Target Problems	5 problems

#	Problem Name	Difficulty
56	Merge Intervals	Medium
179	Largest Number	Medium
252	Meeting Rooms	Easy
315	Count of Smaller Numbers After Self	Hard
937	Reorder Data in Log Files	Medium

Collections Module

Why Important	collections.Counter, deque, defaultdict are interview power tools. They turn O(n) boilerplate into single-line elegance.
Python Concepts	Counter(), Counter.most_common(), defaultdict(list), defaultdict(int), deque(), deque.appendleft(), deque.popleft(), OrderedDict
Target Problems	5 problems

#	Problem Name	Difficulty
347	Top K Frequent Elements	Medium
451	Sort Characters By Frequency	Medium
692	Top K Frequent Words	Medium
239	Sliding Window Maximum	Hard
146	LRU Cache	Medium

Heapq (Priority Queue)

Why Important	heapq is Python's min-heap. Used for Kth largest/smallest, merge K lists, Dijkstra, and scheduling problems.
Python Concepts	heapq.heappush(), heapq.heappop(), heapq.heapify(), heapq.nlargest(), heapq.nsmallest(), max-heap trick with negation
Target Problems	5 problems

#	Problem Name	Difficulty
215	Kth Largest Element in Array	Medium
347	Top K Frequent Elements	Medium
703	Kth Largest Element in Stream	Easy
973	K Closest Points to Origin	Medium
1337	The K Weakest Rows in a Matrix	Easy

Lambda, Comprehensions & Functional Tools

Why Important	Concise Python is expected in interviews. map(), filter(), reduce(), comprehensions shorten code and show Python fluency.
Python Concepts	lambda, list/dict/set comprehension, map(), filter(), zip(), enumerate(), functools.reduce(), any(), all()
Target Problems	4 problems

#	Problem Name	Difficulty
1	Two Sum (one-liner variant)	Easy
905	Sort Array By Parity	Easy
977	Squares of a Sorted Array	Easy
1470	Shuffle the Array	Easy

PHASE 2 — BEGINNER DSA PATTERNS

Learn the six most fundamental problem-solving patterns. Every subsequent phase builds on these. Spend 3–4 weeks here until you can recognise patterns immediately.

Arrays – Core Patterns

Why Important	Arrays are the most tested topic in ALL coding interviews. Master index manipulation, in-place modification, and frequency counting.
Python Concepts	Indexing, two-pointer (same array), in-place modification, prefix products, cumulative operations
Target Problems	12 problems

#	Problem Name	Difficulty
1	Two Sum	Easy
26	Remove Duplicates from Sorted Array	Easy
27	Remove Element	Easy
66	Plus One	Easy
88	Merge Sorted Array	Easy
121	Best Time to Buy and Sell Stock	Easy
169	Majority Element	Easy
189	Rotate Array	Medium
238	Product of Array Except Self	Medium
283	Move Zeroes	Easy
448	Find All Numbers Disappeared	Easy
724	Find Pivot Index	Easy

Strings – Core Patterns

Why Important	String problems test character-level logic, anagram detection, palindromes, and parsing. Common in Google, Amazon, and startup interviews.
Python Concepts	Character frequency, sliding window on strings, two-pointer on strings, ASCII arithmetic, string building
Target Problems	10 problems

#	Problem Name	Difficulty
13	Roman to Integer	Easy
14	Longest Common Prefix	Easy
20	Valid Parentheses	Easy
125	Valid Palindrome	Easy
242	Valid Anagram	Easy
344	Reverse String	Easy

#	Problem Name	Difficulty
387	First Unique Character	Easy
409	Longest Palindrome	Easy
459	Repeated Substring Pattern	Easy
680	Valid Palindrome II	Easy

Hash Maps – Frequency & Lookup

Why Important	Hash maps reduce many $O(n^2)$ brute-force solutions to $O(n)$. They are the #1 optimisation tool for freshers.
Python Concepts	Counter, defaultdict, complement lookups, frequency maps, anagram grouping, two-pass hash maps
Target Problems	10 problems

#	Problem Name	Difficulty
1	Two Sum	Easy
49	Group Anagrams	Medium
128	Longest Consecutive Sequence	Medium
202	Happy Number	Easy
205	Isomorphic Strings	Easy
217	Contains Duplicate	Easy
290	Word Pattern	Easy
349	Intersection of Two Arrays	Easy
383	Ransom Note	Easy
560	Subarray Sum Equals K	Medium

Two Pointers

Why Important	Two pointers eliminate nested loops by moving left/right pointers intelligently. Used in sorted arrays, palindromes, and linked lists.
Python Concepts	Left/right pointer, slow/fast pointer, opposite direction traversal, sorted array assumptions
Target Problems	10 problems

#	Problem Name	Difficulty
11	Container With Most Water	Medium
15	Three Sum	Medium
16	Three Sum Closest	Medium
125	Valid Palindrome	Easy
167	Two Sum II – Input Array Is Sorted	Medium
283	Move Zeroes	Easy
344	Reverse String	Easy
345	Reverse Vowels of a String	Easy

#	Problem Name	Difficulty
532	K-diff Pairs in Array	Medium
977	Squares of a Sorted Array	Easy

Sliding Window

Why Important	Sliding window converts $O(n^2)$ substring/subarray searches to $O(n)$. It appears in ~15% of medium array/string problems.
Python Concepts	Fixed window, variable window, two-pointer window, character frequency window, max/min window tracking
Target Problems	10 problems

#	Problem Name	Difficulty
3	Longest Substring Without Repeating Chars	Medium
121	Best Time to Buy and Sell Stock	Easy
209	Minimum Size Subarray Sum	Medium
219	Contains Duplicate II	Easy
239	Sliding Window Maximum	Hard
424	Longest Repeating Character Replacement	Medium
438	Find All Anagrams in a String	Medium
567	Permutation in String	Medium
643	Maximum Average Subarray I	Easy
1004	Max Consecutive Ones III	Medium

Prefix Sum

Why Important	Prefix sums precompute cumulative totals to answer range sum queries in $O(1)$. Foundation for DP and 2D matrix problems.
Python Concepts	Cumulative sum array, range query $[l, r] = \text{prefix}[r] - \text{prefix}[l-1]$, prefix sum + hash map for subarray problems
Target Problems	8 problems

#	Problem Name	Difficulty
303	Range Sum Query – Immutable	Easy
304	Range Sum Query 2D – Immutable	Medium
307	Range Sum Query – Mutable	Medium
523	Continuous Subarray Sum	Medium
525	Contiguous Array	Medium
560	Subarray Sum Equals K	Medium
724	Find Pivot Index	Easy
1480	Running Sum of 1D Array	Easy

PHASE 3 — INTERMEDIATE DSA

Build your data-structure toolkit: stacks, queues, linked lists, binary search, recursion, and backtracking. These topics separate candidates who can solve Easy problems from those who can crack Medium interviews.

Stack

Why Important	Stacks power bracket matching, expression evaluation, undo/redo, monotonic stack patterns, and DFS iteratively.
Python Concepts	list as stack (append/pop), collections.deque, monotonic stack (increasing/decreasing), call stack awareness
Target Problems	10 problems

#	Problem Name	Difficulty
20	Valid Parentheses	Easy
84	Largest Rectangle in Histogram	Hard
150	Evaluate Reverse Polish Notation	Medium
155	Min Stack	Medium
225	Implement Stack using Queues	Easy
232	Implement Queue using Stacks	Easy
496	Next Greater Element I	Easy
503	Next Greater Element II	Medium
739	Daily Temperatures	Medium
853	Car Fleet	Medium

Queue & Deque

Why Important	Queues drive BFS, level-order traversal, and sliding window maximum. Deque (double-ended queue) is the Python implementation to master.
Python Concepts	collections.deque, FIFO, BFS queue, circular queue, monotonic deque for sliding window
Target Problems	6 problems

#	Problem Name	Difficulty
225	Implement Stack using Queues	Easy
232	Implement Queue using Stacks	Easy
239	Sliding Window Maximum	Hard
622	Design Circular Queue	Medium
641	Design Circular Deque	Medium
933	Number of Recent Calls	Easy

Linked List

Why Important	Linked list questions test pointer manipulation. Fast/slow pointer and dummy-node patterns appear in 70% of linked-list interview problems.
Python Concepts	ListNode class, dummy head, slow/fast pointers, reversing in-place, cycle detection (Floyd's algorithm)
Target Problems	10 problems

#	Problem Name	Difficulty
21	Merge Two Sorted Lists	Easy
83	Remove Duplicates from Sorted List	Easy
141	Linked List Cycle	Easy
142	Linked List Cycle II	Medium
160	Intersection of Two Linked Lists	Easy
206	Reverse Linked List	Easy
234	Palindrome Linked List	Easy
237	Delete Node in a Linked List	Medium
876	Middle of the Linked List	Easy
2	Add Two Numbers	Medium

Binary Search

Why Important	Binary search cuts search space in half each step ($O(\log n)$). Applied not just to arrays but to answer spaces and monotonic functions.
Python Concepts	lo/hi/mid pointers, bisect module (bisect_left, bisect_right), search on answer space, invariant design
Target Problems	10 problems

#	Problem Name	Difficulty
35	Search Insert Position	Easy
69	Sqrt(x)	Easy
153	Find Minimum in Rotated Sorted Array	Medium
162	Find Peak Element	Medium
278	First Bad Version	Easy
374	Guess Number Higher or Lower	Easy
704	Binary Search	Easy
744	Find Smallest Letter Greater Than Target	Easy
875	Koko Eating Bananas	Medium
1011	Capacity to Ship Packages in D Days	Medium

Recursion & Divide and Conquer

Why Important	Recursion is the foundation of trees, graphs, backtracking, and DP. Understanding the call stack and base cases is non-negotiable.
---------------	--

Python Concepts	Base case, recursive case, stack frames, memoisation intro, divide and conquer strategy, merge sort, quick sort
Target Problems	8 problems

#	Problem Name	Difficulty
50	Pow(x, n)	Medium
169	Majority Element (divide & conquer)	Easy
206	Reverse Linked List (recursive)	Easy
226	Invert Binary Tree	Easy
344	Reverse String (recursive)	Easy
509	Fibonacci Number	Easy
779	K-th Symbol in Grammar	Medium
912	Sort an Array (merge sort)	Medium

Backtracking

Why Important	Backtracking explores all possibilities via DFS with pruning. Subsets, permutations, combinations, and N-Queens all use this pattern.
Python Concepts	DFS + undo (backtrack), state space tree, pruning conditions, choose/explore/unchoose template
Target Problems	8 problems

#	Problem Name	Difficulty
17	Letter Combinations of a Phone Number	Medium
22	Generate Parentheses	Medium
39	Combination Sum	Medium
40	Combination Sum II	Medium
46	Permutations	Medium
78	Subsets	Medium
79	Word Search	Medium
90	Subsets II	Medium

PHASE 4 — ADVANCED DSA

Advanced topics that differentiate senior/strong candidates. Trees, Graphs, Greedy, and Dynamic Programming cover 50%+ of medium-hard interview questions at top companies. Take your time — this phase takes 8–10 weeks.

Binary Trees – Traversal & Properties

Why Important	Tree problems appear in nearly every technical interview. Inorder, preorder, postorder, BFS level-order — you must know all four.
Python Concepts	TreeNode class, recursive DFS, iterative BFS (deque), height/depth/diameter, lowest common ancestor
Target Problems	12 problems

#	Problem Name	Difficulty
94	Binary Tree Inorder Traversal	Easy
100	Same Tree	Easy
101	Symmetric Tree	Easy
102	Binary Tree Level Order Traversal	Medium
104	Maximum Depth of Binary Tree	Easy
110	Balanced Binary Tree	Easy
112	Path Sum	Easy
226	Invert Binary Tree	Easy
236	Lowest Common Ancestor of Binary Tree	Medium
257	Binary Tree Paths	Easy
543	Diameter of Binary Tree	Easy
572	Subtree of Another Tree	Easy

Binary Search Tree (BST)

Why Important	BST properties (left < root < right) enable $O(\log n)$ search, insert, and delete. Inorder traversal gives sorted output.
Python Concepts	BST invariant, inorder = sorted, validate BST, successor/predecessor, BST insertion/deletion
Target Problems	8 problems

#	Problem Name	Difficulty
98	Validate Binary Search Tree	Medium
173	BST Iterator	Medium
230	Kth Smallest Element in BST	Medium
235	Lowest Common Ancestor of BST	Medium
450	Delete Node in BST	Medium
700	Search in BST	Easy

#	Problem Name	Difficulty
701	Insert into BST	Medium
938	Range Sum of BST	Easy

Heaps & Priority Queue

Why Important	Heaps are optimal for 'top K', 'running median', and scheduling. Python's heapq (min-heap) is the tool of choice.
Python Concepts	Min-heap, max-heap (negate), heappush/heappop/heapify, heap of tuples, two-heap pattern for median
Target Problems	8 problems

#	Problem Name	Difficulty
215	Kth Largest Element in Array	Medium
295	Find Median from Data Stream	Hard
347	Top K Frequent Elements	Medium
373	Find K Pairs with Smallest Sums	Medium
378	Kth Smallest in Sorted Matrix	Medium
621	Task Scheduler	Medium
703	Kth Largest Element in Stream	Easy
973	K Closest Points to Origin	Medium

Graphs – BFS

Why Important	BFS finds shortest paths in unweighted graphs and is used for level-order traversal, rotten oranges, word ladder problems.
Python Concepts	Adjacency list, visited set, deque BFS, multi-source BFS, BFS on grid (4-directional), shortest path
Target Problems	10 problems

#	Problem Name	Difficulty
102	Binary Tree Level Order Traversal	Medium
127	Word Ladder	Hard
200	Number of Islands	Medium
207	Course Schedule	Medium
210	Course Schedule II	Medium
286	Walls and Gates	Medium
542	01 Matrix	Medium
695	Max Area of Island	Medium
733	Flood Fill	Easy
994	Rotting Oranges	Medium

Graphs – DFS

Why Important	DFS explores all paths, detects cycles, finds connected components, and underpins topological sort and SCCs.
Python Concepts	Recursive DFS, iterative DFS with stack, visited/path sets, cycle detection (colour states), topological sort
Target Problems	10 problems

#	Problem Name	Difficulty
130	Surrounded Regions	Medium
133	Clone Graph	Medium
200	Number of Islands	Medium
207	Course Schedule	Medium
323	Number of Connected Components	Medium
417	Pacific Atlantic Water Flow	Medium
547	Number of Provinces	Medium
695	Max Area of Island	Medium
733	Flood Fill	Easy
797	All Paths From Source to Target	Medium

Greedy Algorithms

Why Important	Greedy makes the locally optimal choice at each step. Used in interval scheduling, jump game, and resource allocation problems.
Python Concepts	Greedy choice property, proof by exchange argument, sorting + greedy, interval greedy, two-pass greedy
Target Problems	8 problems

#	Problem Name	Difficulty
45	Jump Game II	Medium
55	Jump Game	Medium
122	Best Time to Buy/Sell Stock II	Medium
134	Gas Station	Medium
135	Candy	Hard
435	Non-overlapping Intervals	Medium
455	Assign Cookies	Easy
763	Partition Labels	Medium

Dynamic Programming – 1D

Why Important	DP is the most-asked topic at FAANG. 1D DP covers Fibonacci variants, house robber, climbing stairs, and coin change.
Python Concepts	Memoisation (top-down), tabulation (bottom-up), state definition, transition equation, base cases
Target Problems	12 problems

#	Problem Name	Difficulty
70	Climbing Stairs	Easy
118	Pascal's Triangle	Easy
119	Pascal's Triangle II	Easy
121	Best Time to Buy and Sell Stock	Easy
139	Word Break	Medium
198	House Robber	Medium
213	House Robber II	Medium
300	Longest Increasing Subsequence	Medium
322	Coin Change	Medium
338	Counting Bits	Easy
509	Fibonacci Number	Easy
746	Min Cost Climbing Stairs	Easy

Dynamic Programming – 2D

Why Important	2D DP handles grid paths, string alignment, and knapsack variants. Essential for ML interview coding rounds at AI companies.
Python Concepts	2D dp table, state = (i, j), LCS, edit distance, knapsack, grid path DP
Target Problems	8 problems

#	Problem Name	Difficulty
62	Unique Paths	Medium
63	Unique Paths II	Medium
64	Minimum Path Sum	Medium
72	Edit Distance	Medium
97	Interleaving String	Medium
115	Distinct Subsequences	Hard
1143	Longest Common Subsequence	Medium
221	Maximal Square	Medium

6. WEEKLY STUDY SCHEDULE

Follow this daily schedule consistently. Weekends are lighter to avoid burnout. Adjust ML topics based on your current course syllabus.

Day	Time Slot	Activity	Duration
Mon	7:00 – 8:00 AM	LeetCode – New Problem (current phase topic)	60 min
Mon	8:00 – 9:00 AM	Machine Learning – Theory / Video Lecture	60 min
Mon	9:00 – 9:30 AM	Review yesterday's solution & note patterns	30 min
Tue	7:00 – 8:00 AM	LeetCode – New Problem (same topic)	60 min
Tue	8:00 – 9:00 AM	ML – Implement concept in Jupyter Notebook	60 min
Tue	9:00 – 9:30 AM	Read editorial / discuss solution	30 min
Wed	7:00 – 8:00 AM	LeetCode – 2 Easy revision problems	60 min
Wed	8:00 – 9:00 AM	ML – Dataset exploration with Pandas/NumPy	60 min
Wed	9:00 – 9:30 AM	Update Progress Tracker	30 min
Thu	7:00 – 8:00 AM	LeetCode – New Problem (next topic)	60 min
Thu	8:00 – 9:00 AM	ML – Build / train a model (Scikit-Learn)	60 min
Thu	9:00 – 9:30 AM	Write pseudocode for solved problems	30 min
Fri	7:00 – 8:00 AM	LeetCode – Medium problem attempt	60 min
Fri	8:00 – 9:00 AM	ML – Evaluate & visualise model results	60 min
Fri	9:00 – 9:30 AM	Weekly self-assessment	30 min
Sat	9:00 – 10:00 AM	LeetCode – Revision (weakest topic this week)	60 min
Sat	10:00 – 11:00 AM	ML – Mini project or Kaggle notebook	60 min
Sun	10:00 – 11:00 AM	Mock interview (1 problem timed, 30 min)	60 min
Sun	11:00 – 11:30 AM	Plan next week's topics & update tracker	30 min

7. PROGRESS TRACKER

Print this page and tick off topics as you complete them.

Phase / Topic	Assigned	Solved	Revision	Status
PHASE 1 – Python Foundations				
Lists & List Operations	8	/ 8	■	■ Not Started ■ In Progress ■ Done
Strings	8	/ 8	■	■ Not Started ■ In Progress ■ Done
Dictionaries & Hash Maps	6	/ 6	■	■ Not Started ■ In Progress ■ Done
Sets	4	/ 4	■	■ Not Started ■ In Progress ■ Done
Tuples & Sorting	5	/ 5	■	■ Not Started ■ In Progress ■ Done
Collections Module	5	/ 5	■	■ Not Started ■ In Progress ■ Done
Heapq	5	/ 5	■	■ Not Started ■ In Progress ■ Done
Lambda & Comprehensions	4	/ 4	■	■ Not Started ■ In Progress ■ Done
PHASE 2 – Beginner DSA				
Arrays – Core Patterns	12	/ 12	■	■ Not Started ■ In Progress ■ Done
Strings – Core Patterns	10	/ 10	■	■ Not Started ■ In Progress ■ Done
Hash Maps	10	/ 10	■	■ Not Started ■ In Progress ■ Done
Two Pointers	10	/ 10	■	■ Not Started ■ In Progress ■ Done
Sliding Window	10	/ 10	■	■ Not Started ■ In Progress ■ Done
Prefix Sum	8	/ 8	■	■ Not Started ■ In Progress ■ Done
PHASE 3 – Intermediate DSA				
Stack	10	/ 10	■	■ Not Started ■ In Progress ■ Done
Queue & Deque	6	/ 6	■	■ Not Started ■ In Progress ■ Done
Linked List	10	/ 10	■	■ Not Started ■ In Progress ■ Done
Binary Search	10	/ 10	■	■ Not Started ■ In Progress ■ Done

Phase / Topic	Assigned	Solved	Revision	Status
Recursion	8	/ 8	■	■ Not Started ■ In Progress ■ Done
Backtracking	8	/ 8	■	■ Not Started ■ In Progress ■ Done
PHASE 4 – Advanced DSA				
Binary Trees	12	/ 12	■	■ Not Started ■ In Progress ■ Done
BST	8	/ 8	■	■ Not Started ■ In Progress ■ Done
Heaps	8	/ 8	■	■ Not Started ■ In Progress ■ Done
Graphs – BFS	10	/ 10	■	■ Not Started ■ In Progress ■ Done
Graphs – DFS	10	/ 10	■	■ Not Started ■ In Progress ■ Done
Greedy	8	/ 8	■	■ Not Started ■ In Progress ■ Done
DP – 1D	12	/ 12	■	■ Not Started ■ In Progress ■ Done
DP – 2D	8	/ 8	■	■ Not Started ■ In Progress ■ Done

8. MACHINE LEARNING PARALLEL ROADMAP

Study these ML topics in parallel with your DSA phases. Each row below maps to a DSA phase so you can time them together.

DSA Phase	ML Topic	Library / Tool	Key Concepts	Mini Project
Phase 1 (Weeks 1–3)	NumPy & Pandas	numpy, pandas	Arrays, DataFrames, indexing, boolean masking, aggregations	Explore Titanic dataset
Phase 1 (Weeks 2–4)	Data Visualisation	matplotlib, seaborn	Plots: line, bar, scatter, heatmap, histogram	EDA on Iris dataset
Phase 2 (Weeks 4–6)	Supervised Learning Basics	scikit-learn	train_test_split, fit(), predict(), accuracy_score	Predict house prices
Phase 2 (Weeks 5–7)	Linear & Logistic Regression	scikit-learn	Cost function, gradient descent, sigmoid, classification report	Spam classifier
Phase 2 (Weeks 6–8)	KNN & Decision Tree	scikit-learn	Distance metrics, Euclidean, entropy, Gini index, pruning	Iris classification
Phase 3 (Weeks 8–10)	Ensemble Methods	scikit-learn	Random Forest, bagging, feature importance, cross-validation	Credit risk model
Phase 3 (Weeks 9–11)	SVM & Kernel Trick	scikit-learn	Hyperplane, margin, C parameter, RBF kernel, GridSearchCV	Handwriting recognition
Phase 4 (Weeks 11–14)	Unsupervised Learning	scikit-learn	K-Means, elbow method, PCA, dimensionality reduction, silhouette	Customer segmentation
Phase 4 (Weeks 14–18)	Intro to Deep Learning	TensorFlow / PyTorch	Perceptron, activation functions, backprop, epochs, Adam optimizer	MNIST digit classifier
Phase 4 (Weeks 16–20)	Model Evaluation & MLOps	mlflow, joblib	ROC-AUC, confusion matrix, overfitting, pipeline, model saving	End-to-end ML pipeline

9. MATHEMATICS GUIDE FOR AI/ML FRESHERS

Topic	Sub-topics	Level	Used In	Priority
Linear Algebra	Vectors, matrices, dot product, matrix multiply, eigenvalues, SVD	Intermediate	PCA, Neural Networks, SVM, embeddings	MANDATORY
Calculus	Derivatives, chain rule, partial derivatives, gradients	Basic–Intermediate	Gradient descent, backpropagation	MANDATORY
Probability	Bayes theorem, distributions (Normal, Bernoulli), expectation, variance	Intermediate	Naive Bayes, probabilistic models, evaluation	MANDATORY
Statistics	Mean, median, std dev, correlation, hypothesis testing, p-value	Basic	EDA, feature selection, A/B testing	MANDATORY
Discrete Maths	Logic, sets, graphs, trees, modular arithmetic, combinatorics	Basic	Algorithms, hash functions, DSA	MANDATORY
Multivariable Calculus	Jacobians, Hessians, Lagrangians	Advanced	Advanced optimisation	OPTIONAL
Information Theory	Entropy, KL-divergence, cross-entropy	Intermediate	Decision trees, loss functions	OPTIONAL
Numerical Methods	Numerical integration, stability, floating point	Advanced	Scientific computing	OPTIONAL
Abstract Algebra	Groups, rings, fields	Advanced	Cryptography, theoretical ML	SKIP for now

Note: For AI/ML fresher interviews, interviewers expect you to explain concepts like gradient descent, matrix multiplication, and Bayes theorem conceptually – NOT to prove theorems on a whiteboard.

10. MILESTONES & CHECKPOINTS

■ 30 PROBLEMS

(Phase 1 complete)

- ✓ Can use lists, dicts, sets, heapq fluently
- ✓ Solved all Easy Phase 1 problems
- ✓ Comfortable with Python's collections module
- ✓ Understand time complexity basics ($O(n)$, $O(\log n)$, $O(n^2)$)

■ 60 PROBLEMS

(Phase 2 complete)

- ✓ Can apply Two Pointers and Sliding Window automatically
- ✓ Solved Hash Map and Prefix Sum problems without hints
- ✓ Comfortable with Medium difficulty arrays and strings
- ✓ Can explain your approach before coding

■ 100 PROBLEMS

(Phase 3 complete)

- ✓ Can implement Stack, Queue, Linked List from scratch
- ✓ Comfortable with binary search on answer spaces
- ✓ Solved backtracking problems (subsets, permutations)
- ✓ Ready for ML/AI internship coding rounds

■ 150 PROBLEMS

(Phase 4 complete)

- ✓ Tree and graph traversals feel natural
- ✓ Can solve 1D DP problems independently
- ✓ Attempted at least 3 Hard problems
- ✓ Ready for SWE / AI Engineer full-time interviews

11. AI ENGINEER PREPARATION PLAN – NEXT 6 MONTHS

Month	DSA Focus	ML Focus	Expected Skills	Checkpoint
Month 1 Weeks 1–4	Phase 1 complete (Python Foundations) Target: 45 problems	NumPy, Pandas, Matplotlib basics, EDA on 2 datasets	Python fluency $O(n)$ / $O(\log n)$ intuition Basic data wrangling	Solve 5 Easy problems timed (< 20 min each)
Month 2 Weeks 5–8	Phase 2 complete (Beginner DSA) Target: 60 problems	Scikit-Learn, Linear & Logistic Regression	Two Pointers, Sliding Window, Hash Map mastery	Solve 3 Medium problems without hints
Month 3 Weeks 9–12	Phase 3 (first half) Stack, Queue, Linked List Target: 85 problems	KNN, Decision Tree, Random Forest, model evaluation	Pointer manipulation, Binary search on answer spaces	Pass 1 mock interview round (online platform)
Month 4 Weeks 13–16	Phase 3 (second half) Binary Search, Recursion, Backtracking Target: 110 problems	SVM, K-Means, PCA, unsupervised learning project	Recursion trees, Backtracking template, ML model pipeline	Apply for ML/AI internships; Clear OA round
Month 5 Weeks 17–20	Phase 4 (first half) Trees, BST, Heaps, Graphs Target: 130 problems	Intro to Deep Learning, PyTorch/ TensorFlow basics, MNIST project	Graph algorithms, Heap patterns, Tree traversals	Solve LeetCode Weekly Contest; Clear technical phone screen
Month 6 Weeks 21–24	Phase 4 (second half) Greedy, DP 1D & 2D Target: 150 problems + 5 mock interviews	End-to-end ML pipeline project, MLflow, model deployment basics	DP pattern recognition, Greedy intuition, System design basics	Ready for SWE / AI Engineer fresher full-time interviews

Interview Readiness Checkpoints

Checkpoint	Requirement	Status
OA (Online Assessment)	Solve 2 Medium problems in 90 min	■
Technical Phone Screen	Explain approach + code in 45 min	■
DSA Round 1	Trees / Graph / DP medium problems	■
DSA Round 2	System design lite / ML coding	■
ML Conceptual Round	Explain 5 algorithms with pros/cons	■
HR / Behavioural	STAR method, projects, why AI/ML	■

You don't need to be perfect. You need to be CONSISTENT.

Solve 1 problem daily. Review. Repeat. In 6 months, you'll be interview-ready.

Best of Luck — Future AI Engineer! ■